Redis and Rate Limiting with Java

Redis

Redis (Remote Dictionary Server) is a source-available, in-memory data store widely used as a distributed key–value database, cache, and message broker, with optional durability features. Its in-memory design enables low-latency reads and writes, making it particularly effective for caching and real-time applications. As the most popular NoSQL database, Redis is also recognized as one of the most widely used databases overall, catering to a variety of high-performance use cases.

Main Redis Data Types

A simple key-value pair

Command:

`SET key value`

Example:

```
> SET bike:1 Deimos
OK
> GET bike:1
"Deimos"
```

Jedis example (Java):

```java
Jedis jedis = new Jedis();
jedis.set("key", "value");
String value = jedis.get("key");
System.out.println(value);
```

Main Redis Data Types

Ordered collection of strings.

Command:

```
LPUSH list value
```

```
LRANGE list 0 -1
```

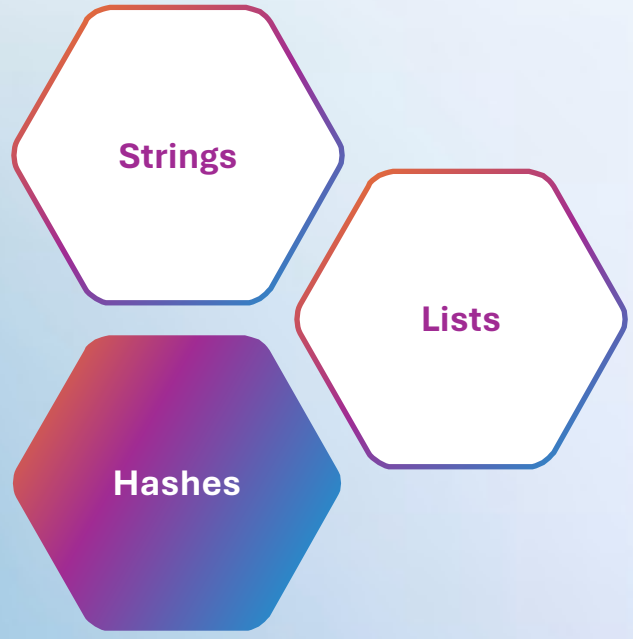
Example:

```
> RPUSH bikes:repairs bike:1 bike:2 bike:3 bike:4 bike:5
(integer) 5
> LTRIM bikes:repairs 0 2
OK
> LRANGE bikes:repairs 0 -1
1) "bike:1"
2) "bike:2"
3) "bike:3"
```

Jedis example (Java):

```java
jedis.lpush("list", "value1", "value2");
List<String> values = jedis.lrange("list", 0, -1);
System.out.println(values);
```

Main Redis Data Types



Key-value pairs within a key.

Command:

```
HSET hash key value
```

```
HGET hash key
```

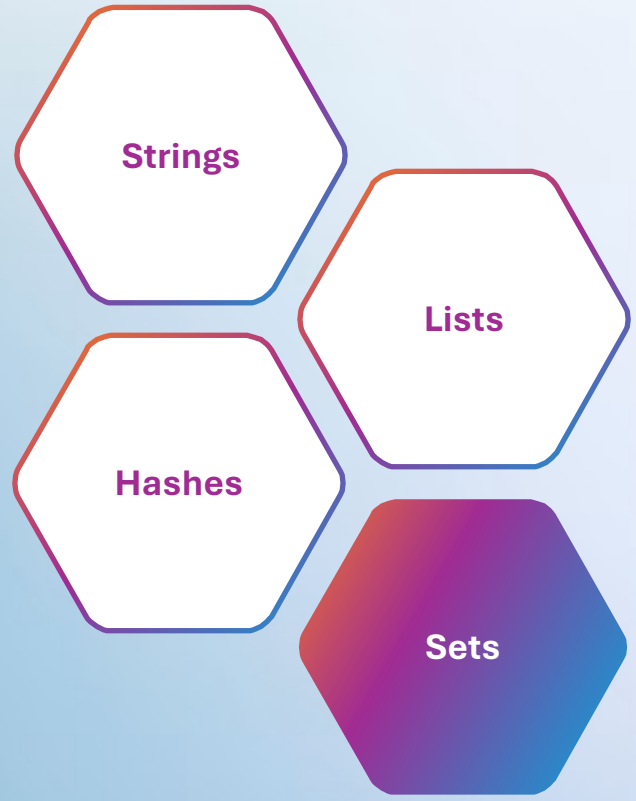
Example:

```
> HSET bike:1 model Deimos brand Ergonom type 'Enduro bikes' price 4972
(integer) 4
> HGET bike:1 model
"Deimos"
> HGET bike:1 price
"4972"
> HGETALL bike:1
1) "model"
2) "Deimos"
3) "brand"
4) "Ergonom"
5) "type"
6) "Enduro bikes"
7) "price"
8) "4972"
```

Jedis example (Java):

```java
jedis.hset("hash", "field", "value");
String fieldValue = jedis.hget("hash", "field");
System.out.println(fieldValue);
```

Main Redis Data Types



Unordered collection of unique strings.

Command:

```
SADD set value
```

```
SMEMBERS set
```

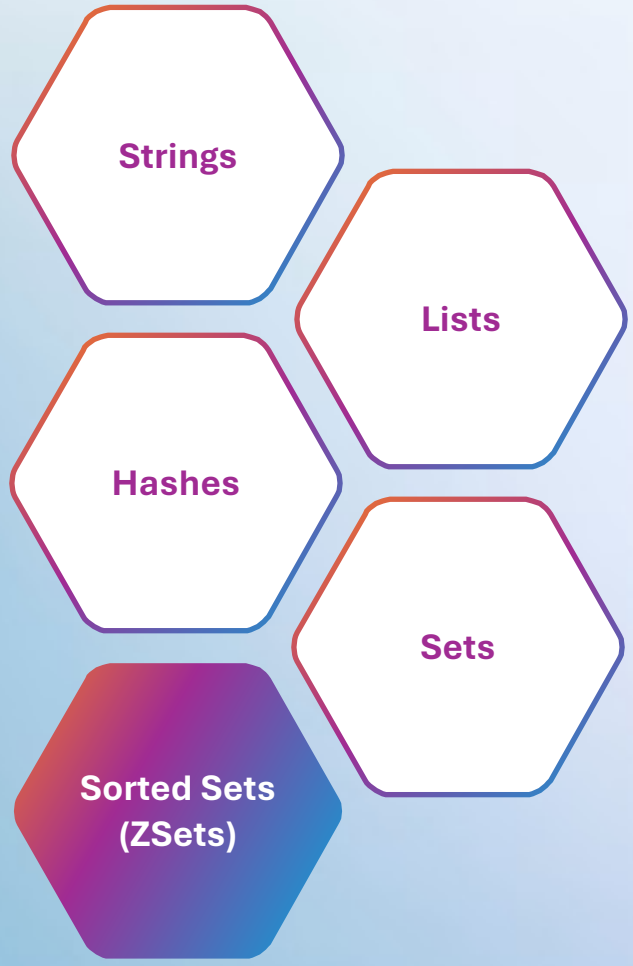
Example:

```
> SADD bikes:racing:france bike:1 bike:2 bike:3
(integer) 3
> SMEMBERS bikes:racing:france
1) bike:3
2) bike:1
3) bike:2
```

Jedis example (Java):

```java
jedis.sadd("set", "value1", "value2");
Set<String> members = jedis.smembers("set");
System.out.println(members);
```

Main Redis Data Types



Similar to sets but with scores for ordering.

Unordered collection of unique strings.

Command:

```
ZADD zset score value
```

```
ZRANGE zset 0 -1 WITHSCORES
```

Example:

```
> ZADD racer_scores 10 "Norem"
(integer) 1
> ZADD racer_scores 12 "Castilla"
(integer) 1
> ZADD racer_scores 8 "Sam-Bodden" 10 "Royce" 6 "Ford" 14 "Prickett"
(integer) 4
```

Jedis example (Java):

```java
long res1 = jedis.zadd("racer_scores", 10d, "Norem");
System.out.println(res1); // >>> 1

long res2 = jedis.zadd("racer_scores", 12d, "Castilla");
System.out.println(res2); // >>> 1

long res3 = jedis.zadd("racer_scores", new HashMap<String,Double>() {{
  put("Sam-Bodden", 8d);
  put("Royce", 10d);
  put("Ford", 6d);
  put("Prickett", 14d);
  put("Castilla", 12d);
}});
System.out.println(res3); // >>> 4
```

Pipelines and Transactions

Pipelines

Used to reduce the number of round trips to the Redis server.

```
Pipeline pipeline = jedis.pipelined();
pipeline.set("key1", "value1");
pipeline.set("key2", "value2");
pipeline.sync();
```

Transactions

Ensures atomic execution of a series of commands.

```
Transaction transaction = jedis.multi();
transaction.set("key1", "value1");
transaction.set("key2", "value2");
transaction.exec();
```

Lua Scripts in Redis

Lua Scripts in Redis

- Lua scripting allows you to run server-side scripts in Redis for atomic operations.
- Key Benefits:
  - Minimized network round-trips.
  - Atomic execution.

```
local key = KEYS[1]
local value = tonumber(ARGV[1])

redis.call("methodName", ...)

local count = redis.call("methodName", ...)

if count < condition then
...
return 1
else
return 0
end
```

Rate Limiter

What is Rate Limiting

Rate Limiting entails techniques to regulate the number of requests a particular client can make against a networked service. It caps the total number and/or the frequency of requests.

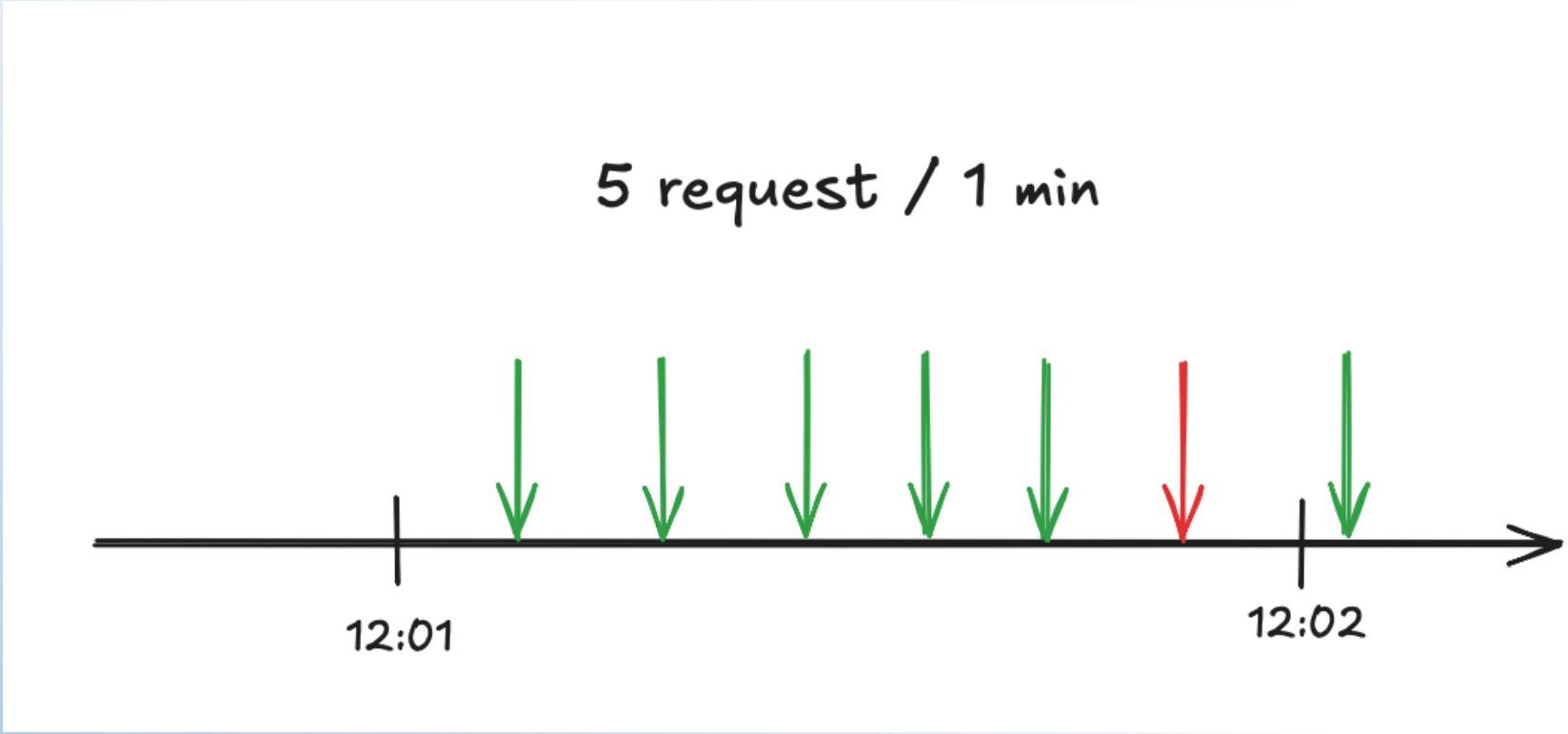
Fixed Window Algorithm

- Divides time into fixed intervals (e.g., 1 minute).
- Pros: Simple implementation.
- Cons: Potential for burst traffic at interval edges.

Useful command in Jedis:

INCR and EXPIRE

Fixed Window Algorithm



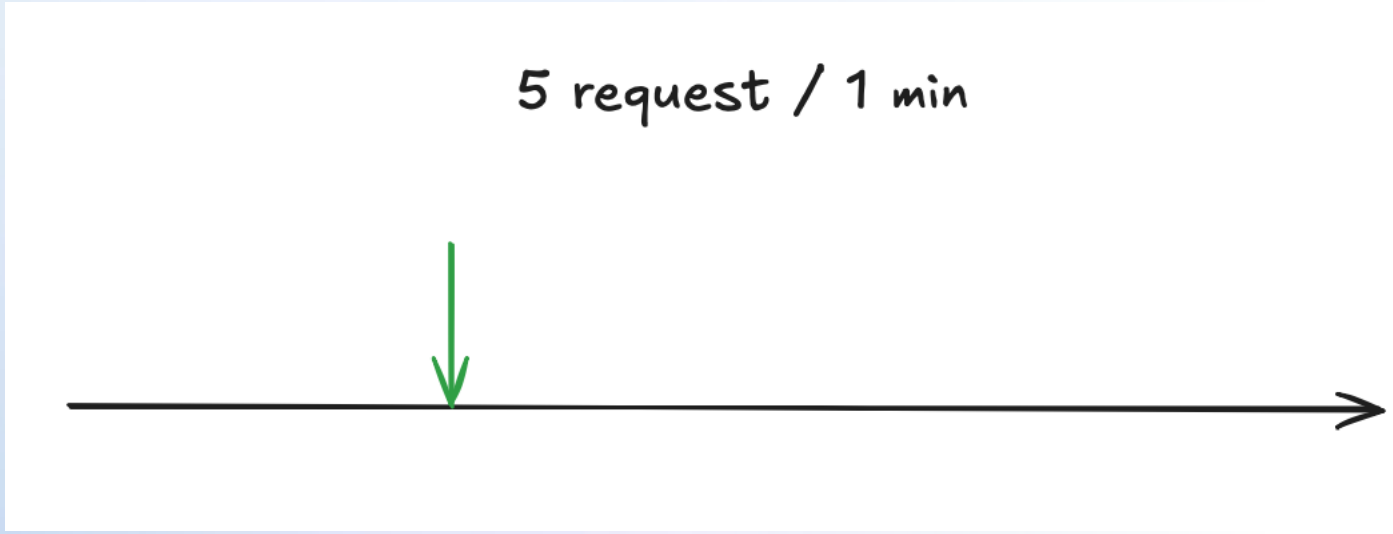
Sliding Window Algorithm

- Allows smoother rate limiting by sliding the window dynamically.
- Uses sorted sets with timestamps.

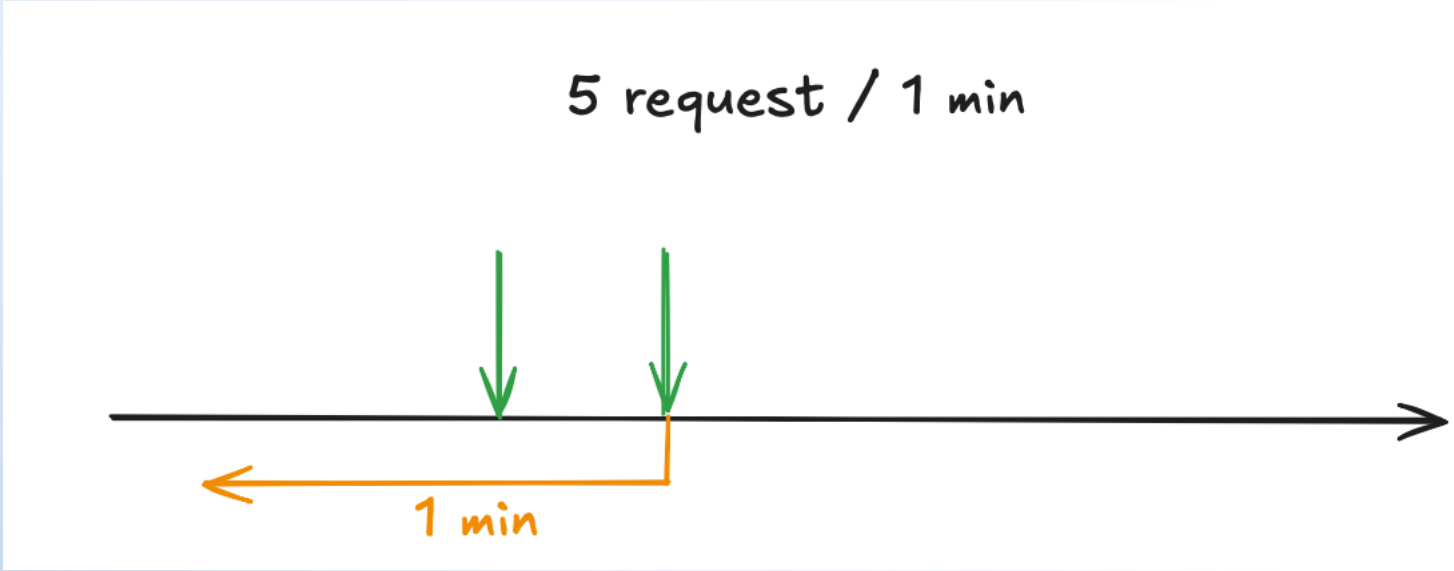
Useful command in Jedis:

ZADD, ZREMRANGEBYSCORE, and ZCARD.

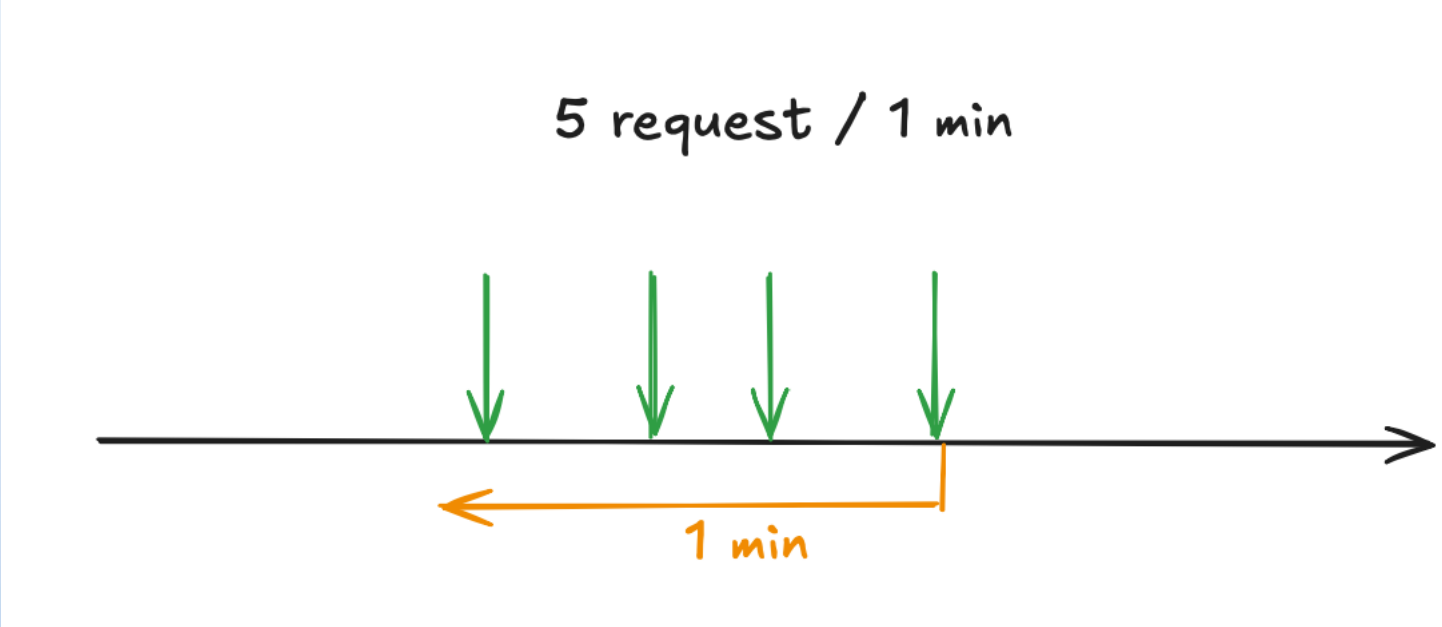
Sliding Window Algorithm



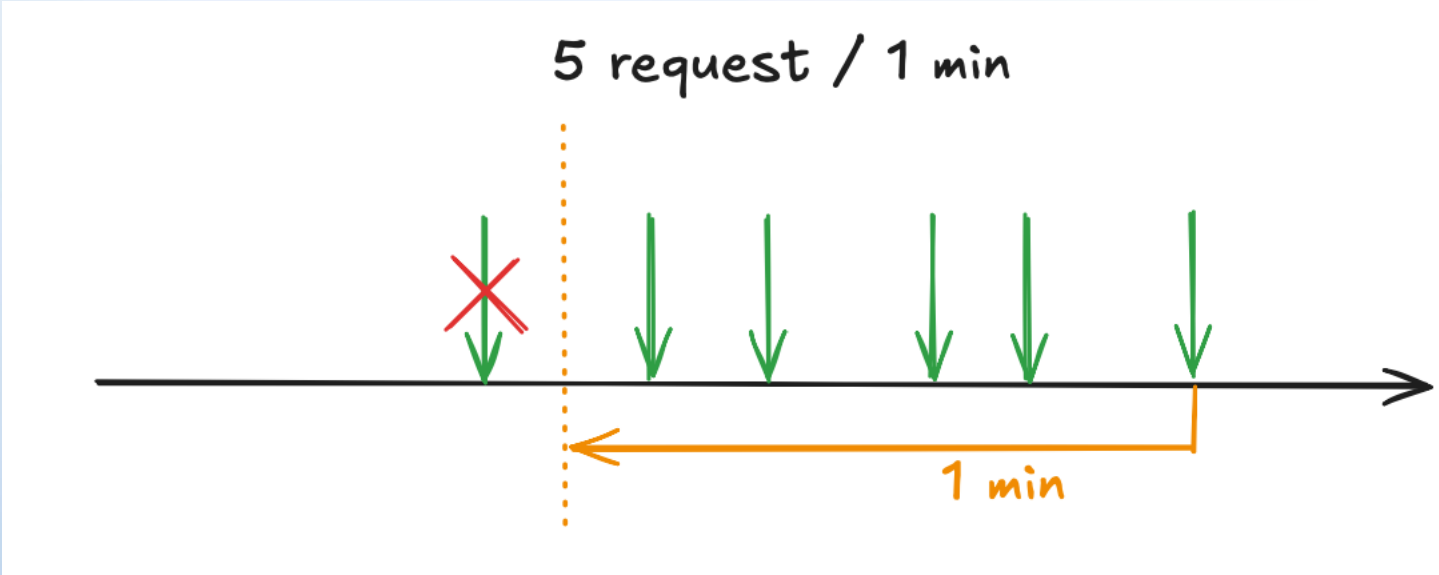
Sliding Window Algorithm



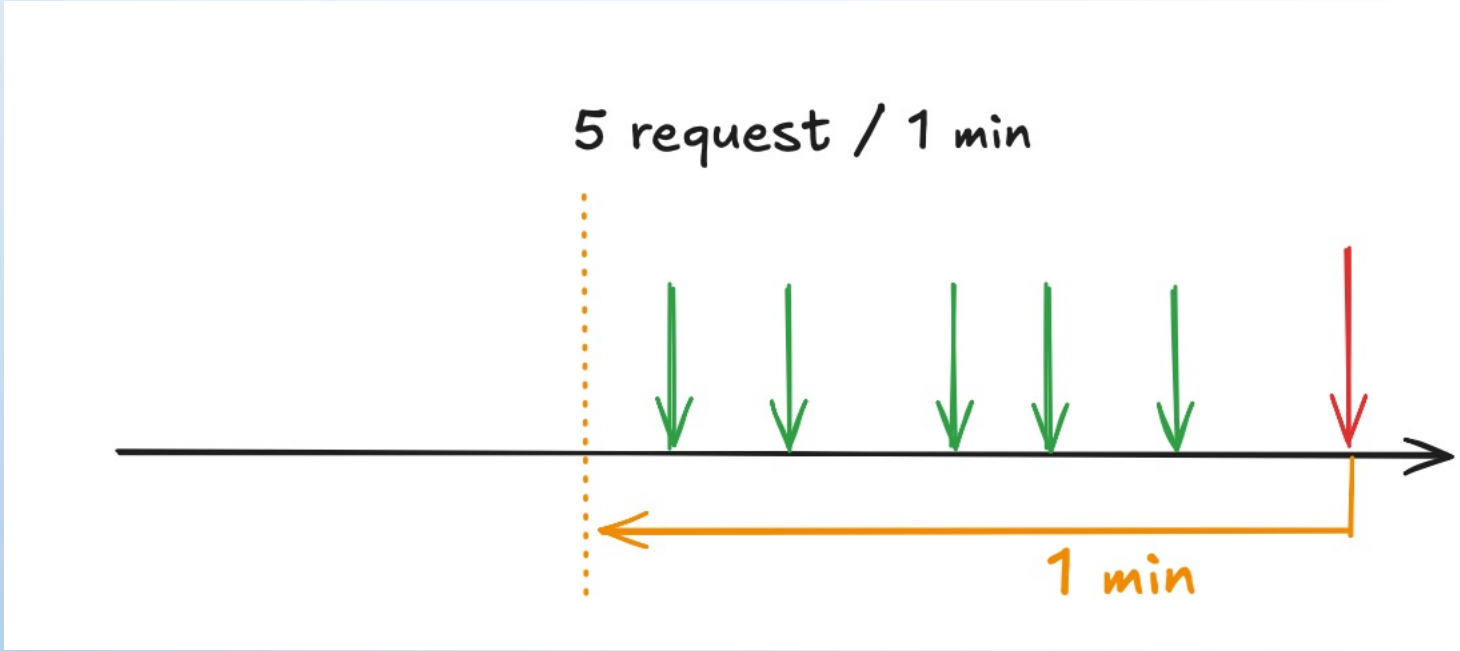
Sliding Window Algorithm



Sliding Window Algorithm



Sliding Window Algorithm



Thank you

- Author: Dmytro Halchenko
- My LinkedIn: https://www.linkedin.com/in/dmytro-halchenko-0307751b6/
- Date: January 2025
- Join Codeus community in Discord
- Join Codeus community in LinkedIn